

Chapter 2

Tutorial lessons 2

2.1 Mathematical functions

MATLAB offers many predefined mathematical functions for technical computing which contains a large set of mathematical functions.

Typing `help elfun` and `help specfun` calls up full lists of *elementary* and *special* functions respectively.

There is a long list of mathematical functions that are *built* into MATLAB. These functions are called *built-ins*. Many standard mathematical functions, such as $\sin(x)$, $\cos(x)$, $\tan(x)$, e^x , $\ln(x)$, are evaluated by the functions `sin`, `cos`, `tan`, `exp`, and `log` respectively in MATLAB.

Table 2.1 lists some commonly used functions, where variables `x` and `y` can be numbers, vectors, or matrices.

Table 2.1: Elementary functions

<code>cos(x)</code>	Cosine	<code>abs(x)</code>	Absolute value
<code>sin(x)</code>	Sine	<code>sign(x)</code>	Signum function
<code>tan(x)</code>	Tangent	<code>max(x)</code>	Maximum value
<code>acos(x)</code>	Arc cosine	<code>min(x)</code>	Minimum value
<code>asin(x)</code>	Arc sine	<code>ceil(x)</code>	Round towards $+\infty$
<code>atan(x)</code>	Arc tangent	<code>floor(x)</code>	Round towards $-\infty$
<code>exp(x)</code>	Exponential	<code>round(x)</code>	Round to nearest integer
<code>sqrt(x)</code>	Square root	<code>rem(x)</code>	Remainder after division
<code>log(x)</code>	Natural logarithm	<code>angle(x)</code>	Phase angle
<code>log10(x)</code>	Common logarithm	<code>conj(x)</code>	Complex conjugate

In addition to the elementary functions, MATLAB includes a number of predefined

constant values. A list of the most common values is given in Table 2.2.

Table 2.2: Predefined constant values

pi	The π number, $\pi = 3.14159\dots$
i, j	The imaginary unit i , $\sqrt{-1}$
Inf	The infinity, ∞
NaN	Not a number

2.1.1 Examples

We illustrate here some typical examples which related to the elementary functions previously defined.

As a first example, the value of the expression $y = e^{-a} \sin(x) + 10\sqrt{y}$, for $a = 5$, $x = 2$, and $y = 8$ is computed by

```
>> a = 5; x = 2; y = 8;
>> y = exp(-a)*sin(x)+10*sqrt(y)
y =
    28.2904
```

The subsequent examples are

```
>> log(142)
ans =
    4.9558

>> log10(142)
ans =
    2.1523
```

Note the difference between the natural logarithm $\log(x)$ and the decimal logarithm (base 10) $\log_{10}(x)$.

To calculate $\sin(\pi/4)$ and e^{10} , we enter the following commands in MATLAB,

```
>> sin(pi/4)
ans =
    0.7071

>> exp(10)
ans =
    2.2026e+004
```

NOTES:

- Only use built-in functions on the right hand side of an expression. Reassigning the value to a built-in function can create problems.
- There are some exceptions. For example, `i` and `j` are pre-assigned to $\sqrt{-1}$. However, one or both of `i` or `j` are often used as loop indices.
- To avoid any possible confusion, it is suggested to use instead `ii` or `jj` as loop indices.

2.2 Basic plotting

2.2.1 overview

MATLAB has an excellent set of graphic tools. Plotting a given data set or the results of computation is possible with very few commands. You are highly encouraged to plot mathematical functions and results of analysis as often as possible. Trying to understand mathematical equations with graphics is an enjoyable and very efficient way of learning mathematics. Being able to plot mathematical functions and data freely is the most important step, and this section is written to assist you to do just that.

2.2.2 Creating simple plots

The basic MATLAB graphing procedure, for example in 2D, is to take a vector of x -coordinates, $\mathbf{x} = (x_1, \dots, x_N)$, and a vector of y -coordinates, $\mathbf{y} = (y_1, \dots, y_N)$, locate the points (x_i, y_i) , with $i = 1, 2, \dots, n$ and then join them by straight lines. You need to prepare x and y in an identical array form; namely, x and y are both row arrays or column arrays of the *same* length.

The MATLAB command to plot a graph is `plot(x,y)`. The vectors $\mathbf{x} = (1, 2, 3, 4, 5, 6)$ and $\mathbf{y} = (3, -1, 2, 4, 5, 1)$ produce the picture shown in Figure 2.1.

```
>> x = [1 2 3 4 5 6];  
>> y = [3 -1 2 4 5 1];  
>> plot(x,y)
```

NOTE: The `plot` functions has different forms depending on the input arguments. If $\mathbf{y}$ is a vector `plot(y)` produces a piecewise linear graph of the elements of $\mathbf{y}$ versus the index of the elements of $\mathbf{y}$. If we specify two vectors, as mentioned above, `plot(x,y)` produces a graph of $\mathbf{y}$ versus $\mathbf{x}$.

For example, to plot the function $\sin(x)$ on the interval $[0, 2\pi]$, we first create a vector of x values ranging from 0 to 2π , then compute the *sine* of these values, and finally plot the result:

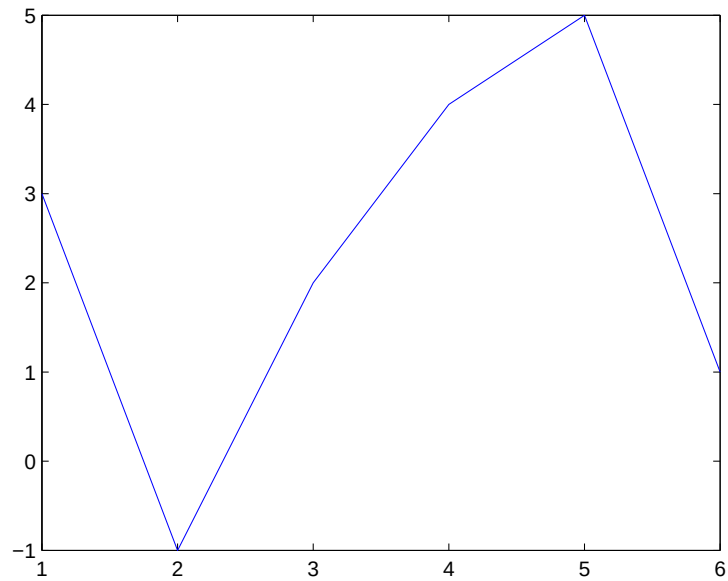


Figure 2.1: Plot for the vectors x and y

```
>> x = 0:pi/100:2*pi;
>> y = sin(x);
>> plot(x,y)
```

NOTES:

- `0:pi/100:2*pi` yields a vector that
 - starts at 0,
 - takes steps (or increments) of $\pi/100$,
 - stops when 2π is reached.
- If you omit the increment, MATLAB automatically increments by 1.

2.2.3 Adding titles, axis labels, and annotations

MATLAB enables you to add axis labels and titles. For example, using the graph from the previous example, add an x - and y -axis labels.

Now label the axes and add a title. The character `\pi` creates the symbol π . An example of 2D plot is shown in Figure 2.2.

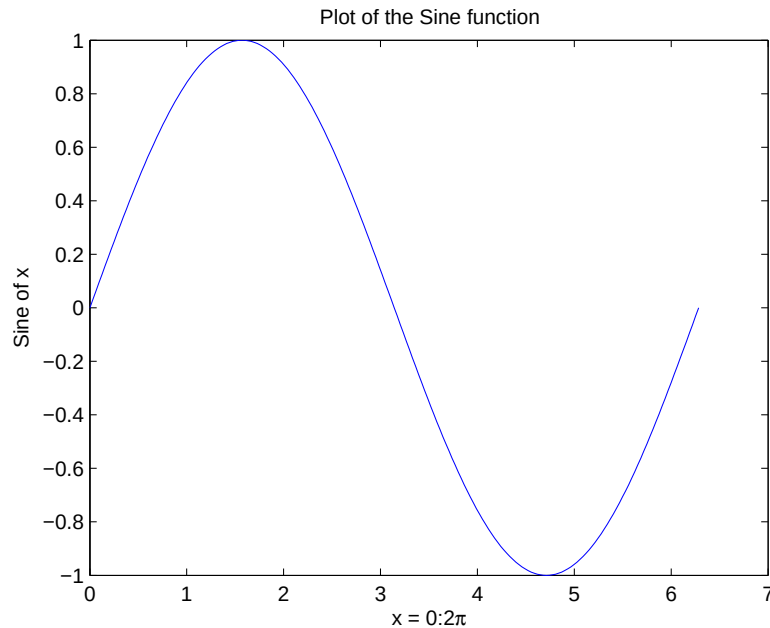


Figure 2.2: Plot of the Sine function

```
>> xlabel('x = 0:2\pi')
>> ylabel('Sine of x')
>> title('Plot of the Sine function')
```

The color of a single curve is, by default, **blue**, but other colors are possible. The desired color is indicated by a third argument. For example, **red** is selected by `plot(x,y,'r')`. Note the single quotes, `' '`, around `r`.

2.2.4 Multiple data sets in one plot

Multiple (x, y) *pairs* arguments create *multiple* graphs with a single call to `plot`. For example, these statements plot three related functions of x : $y_1 = 2\cos(x)$, $y_2 = \cos(x)$, and $y_3 = 0.5\cos(x)$, in the interval $0 \leq x \leq 2\pi$.

```
>> x = 0:pi/100:2*pi;
>> y1 = 2*cos(x);
>> y2 = cos(x);
>> y3 = 0.5*cos(x);
>> plot(x,y1,'--',x,y2,'-',x,y3,':')
>> xlabel('0 \leq x \leq 2\pi')
>> ylabel('Cosine functions')
>> legend('2*cos(x)', 'cos(x)', '0.5*cos(x)')
```

```
>> title('Typical example of multiple plots')
>> axis([0 2*pi -3 3])
```

The result of multiple data sets in one graph plot is shown in Figure 2.3.

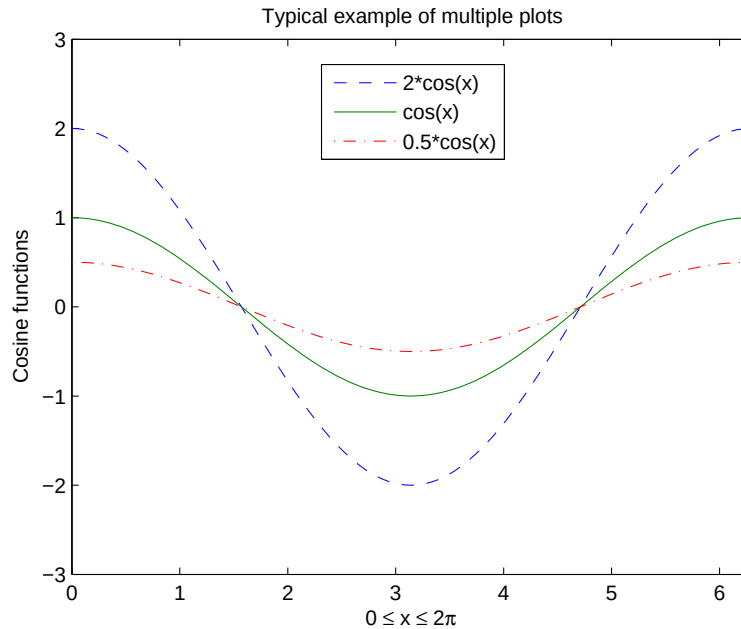


Figure 2.3: Typical example of multiple plots

By default, MATLAB uses *line style* and *color* to distinguish the data sets plotted in the graph. However, you can change the appearance of these graphic components or add annotations to the graph to help explain your data for presentation.

2.2.5 Specifying line styles and colors

It is possible to specify *line styles*, *colors*, and *markers* (e.g., circles, plus signs, ...) using the `plot` command:

```
plot(x,y,'style_color_marker')
```

where `style_color_marker` is a *triplet* of values from Table 2.3.

To find additional information, type `help plot` or `doc plot`.

Table 2.3: Attributes for `plot`

SYMBOL	COLOR	SYMBOL	LINE STYLE	SYMBOL	MARKER
<code>k</code>	Black	<code>—</code>	Solid	<code>+</code>	Plus sign
<code>r</code>	Red	<code>--</code>	Dashed	<code>o</code>	Circle
<code>b</code>	Blue	<code>:</code>	Dotted	<code>*</code>	Asterisk
<code>g</code>	Green	<code>-.</code>	Dash-dot	<code>.</code>	Point
<code>c</code>	Cyan	<code>none</code>	No line	<code>×</code>	Cross
<code>m</code>	Magenta			<code>s</code>	Square
<code>y</code>	Yellow			<code>d</code>	Diamond

2.3 Exercises

NOTE: Due to the teaching class during this Fall Quarter 2005, the *problems* are *temporarily* removed from this section.

2.4 Introduction

Matrices are the basic elements of the MATLAB environment. A matrix is a two-dimensional array consisting of m rows and n columns. Special cases are *column vectors* ($n = 1$) and *row vectors* ($m = 1$).

In this section we will illustrate how to apply different *operations* on matrices. The following topics are discussed: vectors and matrices in MATLAB, the inverse of a matrix, determinants, and matrix manipulation.

MATLAB supports two types of operations, known as *matrix operations* and *array operations*. Matrix operations will be discussed first.

2.5 Matrix generation

Matrices are fundamental to MATLAB. Therefore, we need to become familiar with matrix generation and manipulation. Matrices can be generated in several ways.

2.5.1 Entering a vector

A vector is a special case of a matrix. The purpose of this section is to show how to create vectors and matrices in MATLAB. As discussed earlier, an array of dimension $1 \times n$ is called a *row* vector, whereas an array of dimension $m \times 1$ is called a *column* vector. The elements of vectors in MATLAB are enclosed by square brackets and are separated by spaces or by commas. For example, to enter a row vector, v , type

```
>> v = [1 4 7 10 13]
v =
     1     4     7    10    13
```

Column vectors are created in a similar way, however, semicolon (;) must separate the components of a column vector,

```
>> w = [1;4;7;10;13]
w =
     1
     4
     7
    10
    13
```

On the other hand, a *row* vector is converted to a *column* vector using the *transpose* operator. The *transpose* operation is denoted by an apostrophe or a single quote (').


```
>> w = v'
w =
     1
     4
     7
    10
    13
```

Thus, $v(1)$ is the first element of vector $\mathbf{v}$, $v(2)$ its second element, and so forth.

Furthermore, to access *blocks* of elements, we use MATLAB's colon notation ($:$). For example, to access the first three elements of $\mathbf{v}$, we write,

```
>> v(1:3)
ans =
     1     4     7
```

Or, all elements from the third through the last elements,

```
>> v(3:end)
ans =
     7    10    13
```

where **end** signifies the *last* element in the vector. If $\mathbf{v}$ is a vector, writing

```
>> v(:)
```

produces a column vector, whereas writing

```
>> v(1:end)
```

produces a row vector.

2.5.2 Entering a matrix

A matrix is an array of numbers. To type a matrix into MATLAB you must

- begin with a square bracket, `[`
- separate elements in a row with spaces or commas `(,)`
- use a semicolon `(;)` to separate rows
- end the matrix with another square bracket, `]`.

Here is a typical example. To enter a matrix **A**, such as,

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad (2.1)$$

type,

```
>> A = [1 2 3; 4 5 6; 7 8 9]
```

MATLAB then displays the 3×3 matrix as follows,

```
A      =  
      1      2      3  
      4      5      6  
      7      8      9
```

Note that the use of semicolons (;) here is different from their use mentioned earlier to suppress output or to write multiple commands in a single line.

Once we have entered the matrix, it is automatically stored and remembered in the *Workspace*. We can refer to it simply as matrix **A**. We can then view a particular element in a matrix by specifying its location. We write,

```
>> A(2,1)  
ans =  
      4
```

A(2,1) is an element located in the second row and first column. Its value is 4.

2.5.3 Matrix indexing

We select elements in a matrix just as we did for vectors, but now we need two indices. The element of row i and column j of the matrix **A** is denoted by **A(i,j)**. Thus, **A(i,j)** in MATLAB refers to the element A_{ij} of matrix **A**. The *first* index is the *row* number and the *second* index is the *column* number. For example, **A(1,3)** is an element of *first* row and *third* column. Here, **A(1,3)=3**.

Correcting any entry is easy through indexing. Here we substitute **A(3,3)=9** by **A(3,3)=0**. The result is

```
>> A(3,3) = 0  
A      =  
      1      2      3  
      4      5      6  
      7      8      0
```

Single elements of a matrix are accessed as $A(i, j)$, where $i \geq 1$ and $j \geq 1$. Zero or negative subscripts are not supported in MATLAB.

2.5.4 Colon operator

The colon operator will prove very useful and understanding how it works is the key to efficient and convenient usage of MATLAB. It occurs in several different forms.

Often we must deal with matrices or vectors that are too large to enter one element at a time. For example, suppose we want to enter a vector x consisting of points $(0, 0.1, 0.2, 0.3, \dots, 5)$. We can use the command

```
>> x = 0:0.1:5;
```

The row vector has 51 elements.

2.5.5 Linear spacing

On the other hand, there is a command to generate linearly spaced vectors: `linspace`. It is similar to the colon operator `(:)`, but gives direct control over the number of points. For example,

```
y = linspace(a,b)
```

generates a row vector y of 100 points linearly spaced between and including a and b .

```
y = linspace(a,b,n)
```

generates a row vector y of n points linearly spaced between and including a and b . This is useful when we want to divide an interval into a number of subintervals of the same length. For example,

```
>> theta = linspace(0,2*pi,101)
```

divides the interval $[0, 2\pi]$ into 100 equal subintervals, then creating a vector of 101 elements.

2.5.6 Colon operator in a matrix

The colon operator can also be used to pick out a certain row or column. For example, the statement $A(m:n, k:l)$ specifies rows m to n and column k to l . Subscript expressions refer to portions of a matrix. For example,

```
>> A(2,:)
ans =
    4    5    6
```

is the second row elements of A.

The colon operator can also be used to extract a sub-matrix from a matrix A.

```
>> A(:,2:3)
ans =
    2    3
    5    6
    8    0
```

A(:,2:3) is a sub-matrix with the last two columns of A.

A row or a column of a matrix can be deleted by setting it to a *null* vector, [].

```
>> A(:,2)=[]
ans =
    1    3
    4    6
    7    0
```

2.5.7 Creating a sub-matrix

To extract a *submatrix* B consisting of rows 2 and 3 and columns 1 and 2 of the matrix A, do the following

```
>> B = A([2 3],[1 2])
B =
    4    5
    7    8
```

To interchange rows 1 and 2 of A, use the vector of row indices together with the colon operator.

```
>> C = A([2 1 3], :)
C =
    4    5    6
    1    2    3
    7    8    0
```

It is important to note that the *colon operator* (:) stands for *all columns* or *all rows*. To create a vector version of matrix A, do the following

```
>> A(:)
ans =
     1
     2
     3
     4
     5
     6
     7
     8
     0
```

The submatrix comprising the intersection of rows p to q and columns r to s is denoted by $A(p:q,r:s)$.

As a special case, a colon $(:)$ as the row or column specifier covers all entries in that row or column; thus

- $A(:,j)$ is the j th column of A , while
- $A(i,:)$ is the i th row, and
- $A(\text{end},:)$ picks out the last row of A .

The keyword `end`, used in $A(\text{end},:)$, denotes the last index in the specified dimension. Here are some examples.

```
>> A
A =
     1     2     3
     4     5     6
     7     8     9
```

```
>> A(2:3,2:3)
ans =
     5     6
     8     9
```

```
>> A(end:-1:1,end)
ans =
     9
     6
     3
```

```
>> A([1 3],[2 3])
ans =
     2     3
     8     9
```

2.5.8 Deleting row or column

To delete a row or column of a matrix, use the *empty vector* operator, `[]`.

```
>> A(3,:) = []
A =
     1     2     3
     4     5     6
```

Third row of matrix A is now deleted. To restore the third row, we use a technique for creating a matrix

```
>> A = [A(1,:);A(2,:);[7 8 0]]
A =
     1     2     3
     4     5     6
     7     8     0
```

Matrix A is now restored to its original form.

2.5.9 Dimension

To determine the *dimensions* of a matrix or vector, use the command `size`. For example,

```
>> size(A)
ans =
     3     3
```

means 3 rows and 3 columns.

Or more explicitly with,

```
>> [m,n]=size(A)
```

2.5.10 Continuation

If it is not possible to type the entire input on the same line, use consecutive periods, called an ellipsis ..., to signal continuation, then continue the input on the next line.

```
B = [4/5      7.23*tan(x)      sqrt(6); ...
      1/x^2    0                3/(x*log(x)); ...
      x-7      sqrt(3)          x*sin(x)];
```

Note that *blank* spaces around +, −, = signs are optional, but they improve readability.

2.5.11 Transposing a matrix

The *transpose* operation is denoted by an apostrophe or a single quote ('). It flips a matrix about its main diagonal and it turns a row vector into a column vector. Thus,

```
>> A'
ans =
     1     4     7
     2     5     8
     3     6     0
```

By using linear algebra notation, the transpose of $m \times n$ real matrix **A** is the $n \times m$ matrix that results from interchanging the rows and columns of **A**. The transpose matrix is denoted A^T .

2.5.12 Concatenating matrices

Matrices can be made up of sub-matrices. Here is an example. First, let's recall our previous matrix **A**.

```
A =
     1     2     3
     4     5     6
     7     8     9
```

The new matrix **B** will be,

```
>> B = [A 10*A; -A [1 0 0; 0 1 0; 0 0 1]]
B =
     1     2     3    10    20    30
```

4	5	6	40	50	60
7	8	9	70	80	90
-1	-2	-3	1	0	0
-4	-5	-6	0	1	0
-7	-8	-9	0	0	1

2.5.13 Matrix generators

MATLAB provides functions that generates elementary matrices. The matrix of zeros, the matrix of ones, and the identity matrix are returned by the functions **zeros**, **ones**, and **eye**, respectively.

Table 2.4: Elementary matrices

eye(m,n)	Returns an m-by-n matrix with 1 on the main diagonal
eye(n)	Returns an n-by-n square identity matrix
zeros(m,n)	Returns an m-by-n matrix of zeros
ones(m,n)	Returns an m-by-n matrix of ones
diag(A)	Extracts the diagonal of matrix A
rand(m,n)	Returns an m-by-n matrix of random numbers

For a complete list of *elementary matrices* and *matrix manipulations*, type **help elmat** or **doc elmat**. Here are some examples:

```
1.      >> b=ones(3,1)
      b
      =
      1
      1
      1
```

Equivalently, we can define **b** as **>> b=[1;1;1]**

```
2.      >> eye(3)
      ans
      =
      1     0     0
      0     1     0
      0     0     1
```

```
3.      >> c=zeros(2,3)
      c
      =
      0     0     0
```


0 0 0

In addition, it is important to remember that the three elementary operations of *addition* (+), *subtraction* (−), and *multiplication* (*) apply also to matrices whenever the dimensions are *compatible*.

Two other important matrix generation functions are **rand** and **randn**, which generate matrices of (pseudo-)random numbers using the same syntax as **eye**.

In addition, matrices can be constructed in a block form. With **C** defined by **C** = [1 2; 3 4], we may create a matrix **D** as follows

```
>> D = [C zeros(2); ones(2) eye(2)]
D =
     1     2     0     0
     3     4     0     0
     1     1     1     0
     1     1     0     1
```

2.5.14 Special matrices

MATLAB provides a number of special matrices (see Table 2.5). These matrices have interesting properties that make them useful for constructing examples and for testing algorithms. For more information, see MATLAB documentation.

Table 2.5: Special matrices

hilb	Hilbert matrix
invhilb	Inverse Hilbert matrix
magic	Magic square
pascal	Pascal matrix
toeplitz	Toeplitz matrix
vander	Vandermonde matrix
wilkinson	Wilkinson's eigenvalue test matrix

2.6 Exercises

NOTE: Due to the teaching class during this Fall Quarter 2005, the *problems* are *temporarily* removed from this section.